

Network Policy Validation

Detailed validation of zero-trust Kubernetes pod communication using Calico-enforced NetworkPolicies, test clients, expected outcomes, and CIS Benchmark alignment.

Field	Value
Document Type	Network Security Validation
Author	Jagriti Banerjee
Project Scope	Private Kubernetes DevSecOps lab using Docker, Kind, Calico, Trivy, Falco, OPA/Gatekeeper, ArgoCD, and kube-bench
Audience	Security engineers, DevSecOps reviewers, recruiters, and technical interviewers
Classification	Private lab / portfolio evidence

Reviewer snapshot

Review Dimension	Included in Document
Technical depth	Control rationale, validation commands, expected results, risk interpretation, and remediation guidance.
Evidence model	Screenshot references, manifest fields, scanner outputs, and repeatable kubectl commands.
Professional use	Can be used as a GitHub portfolio artifact, interview talking point, and internal-style security record.
CIS alignment	Includes a dedicated CIS Kubernetes Benchmark section with lab-context interpretation.

Scope statement

This document is based on controlled, authorized testing performed inside a private lab environment. It does not represent a production certification and does not include public or third-party targets. The focus is professional evidence, repeatable validation, and security engineering reasoning.

Spacing and readability note

The document is intentionally laid out with compact paragraphs, split tables, and dedicated evidence pages so that detail is preserved without crowded pages or large blank gaps.

Document coverage

Coverage Area	Included
Assessment narrative	Executive summary, scope, methodology, technical analysis, and validation interpretation.
Evidence references	Relevant screenshots, command outputs, reports, and expected retest outcomes.
Remediation planning	Priority-based actions, production notes, and follow-up recommendations.
Benchmark context	CIS Kubernetes Benchmark alignment and lab-specific interpretation.

1. Executive Summary

This document validates the NetworkPolicy implementation for the Enterprise DevSecOps lab. The goal is to prove that the cluster does not merely contain NetworkPolicy YAML, but actively enforces traffic restrictions with Calico. The validation demonstrates default-deny behavior, allowed trusted ingress, blocked untrusted pods, approved pod labels, and DNS egress handling.

2. Policy Design

Policy	Purpose	Security Value
default-deny-all	Applies podSelector {} with Ingress and Egress policy types.	Blocks communication unless explicitly allowed.
allow-demo-app-ingress	Allows traffic to demo-app only from ingress namespace or pods labelled access=allowed.	Prevents arbitrary pods from reaching app service.
allow-dns-egress	Allows pods to query kube-dns on TCP/UDP 53.	Preserves service discovery while keeping egress controlled.
allow-approved-client-egress-to-app	Allows approved client pods to reach app on port 5000.	Models zero-trust allowlisting.

3. Validation Method

```
kubectl get networkpolicy -n devsecops
```

```
kubectl -n ingress-nginx exec allowed-ingress-curl -- curl -sS --max-time 5  
http://demo-app-svc.devsecops.svc.cluster.local:5000/health
```

```
kubectl -n devsecops exec blocked-curl -- curl -sS --max-time 5 http://demo-app-svc:5000/health
```

```
kubectl -n devsecops exec allowed-curl -- curl -sS --max-time 5 http://demo-app-svc:5000/health
```

4. Test Case Matrix

Test ID	Source	Expected Result	Observed Meaning
NP-01	Unlabelled pod in devsecops	Timeout / blocked	Default-deny policy is effective.
NP-02	Pod with access=allowed label	HTTP 200 / health ok	Approved in-namespace client can connect.
NP-03	Pod in ingress-nginx namespace	HTTP 200 / health ok	Trusted namespace ingress rule works.
NP-04	DNS egress from pods	DNS resolution works	NetworkPolicy does not break service discovery.
NP-05	NetworkPolicy list	All expected policies present	Policy configuration is deployed.

5. CIS Kubernetes Benchmark Section

CIS describes the Kubernetes Benchmark as consensus-based secure configuration guidance for Kubernetes. In this lab, CIS Benchmark alignment is used as an assessment reference and learning framework. Because the cluster is a Kind-based private lab rather than a production kubeadm or managed service environment, benchmark results are interpreted with environment context. The expected professional practice is to document PASS, FAIL, WARN, and not-applicable outcomes; assign owners to applicable failures; retest after remediation; and keep benchmark artifacts in the evidence folder.

NetworkPolicy validation supports CIS-style defense-in-depth by limiting pod-to-pod communication. CIS Benchmark items may not fully validate application-layer network segmentation; therefore, this document includes live positive and negative traffic tests as stronger practical evidence.

6. Evidence Highlights

7. Security Interpretation

- The default-deny policy prevents accidental exposure of the demo application to arbitrary pods.
- The allow rules model production-style trust boundaries around ingress controllers and approved service identities.
- The validation includes negative testing, which is essential because the presence of a NetworkPolicy object alone does not prove enforcement.
- Calico makes this lab stronger because Kind default networking alone may not enforce all NetworkPolicy behavior.
- DNS egress is explicitly permitted to avoid breaking normal Kubernetes service discovery.

8. Retest Checklist

Retest Item	Expected Result
NetworkPolicy objects exist	kubectl get networkpolicy -n devsecops lists default deny and allow policies.
Allowed ingress namespace test	curl to /health returns success.
Untrusted pod test	curl fails or times out.
Approved pod label test	curl to /health returns success.
DNS egress test	Kubernetes service DNS name resolves.

Quality Assurance and Evidence Preservation

QA Item	Required Practice
Evidence naming	Use stable screenshot names and link each item to a control or retest step.
Command repeatability	Store commands in documentation so another reviewer can reproduce validation.
Exception handling	Any accepted risk should include owner, reason, expiry date, and compensating control.
Retest discipline	Do not close a remediation item until the expected negative and positive tests both pass.
CI/CD linkage	Where possible, convert manual validation into security gates or scheduled checks.

Reviewer notes for this Kubernetes security area

This document intentionally combines implementation evidence with risk interpretation. The objective is to show not only that commands were executed, but also why each control matters, how it reduces attack surface, and how the result should be retested after future changes.

References and Standards Basis

Reference	Use in this Document
CIS Kubernetes Benchmark	CIS describes its Kubernetes Benchmark as a product of a community consensus process with secure configuration guidelines for Kubernetes.
Kubernetes RBAC documentation	Kubernetes RBAC uses Role, ClusterRole, RoleBinding, and ClusterRoleBinding objects to manage authorization.
Kubernetes NetworkPolicy documentation	NetworkPolicy governs how pods communicate with each other and with network endpoints.
Kubernetes Pod Security Standards	The restricted profile is intended for strongly hardened workload execution.

Reference URLs used during document preparation: <https://www.cisecurity.org/benchmark/kubernetes>, <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>, <https://kubernetes.io/docs/concepts/services-networking/network-policies/>, and <https://kubernetes.io/docs/concepts/security/pod-security-standards/>.